

Domain Model

ADBO Teaching Team

Even Semester 2024/2025

D4 Informatics Engineering Study Program

Information Technology Department

Learning Outcomes

- After completing this material, students are expected to:
 - Refreshing the understanding of software development flow concepts and methodologies
 - Refreshing the understanding of object-oriented software development paradigms
 - Understanding the basic concepts of domain modeling
 - Understanding the stages of the domain modeling process

Software Development Life Cycle

Software development life cycle

Plot , Roles , and Stages (*Deliverable*) (Tilley, 2012) (Dennis, 2016) (Valacich, 2017)



1. **User/Product Owner** brings software requirement (change) request (**System Request**) to System Analyst
2. **The System Analyst** makes a feasibility analysis of the System Request.
3. After being declared feasible, the System Analyst carries out analysis and design, and the result is **the System Specification**.
 - **Business Analyst** helps System Analyst understand the business processes of the software to be built.
4. System Specification is submitted by the System Analyst to **the Programmer** to do **Construction** (*Coding*)
5. The construction results in the form of **program code** are submitted to **the Software Tester** for **testing** (*Unit, Integration, System, User Acceptance Testing*).
6. Software **installation** (*delivery*) and change management
 - **Software** = Program Code + Documentation (Development and Usage)
7. The cycle returns to 1 if there is a change request (**Software Change Request**)

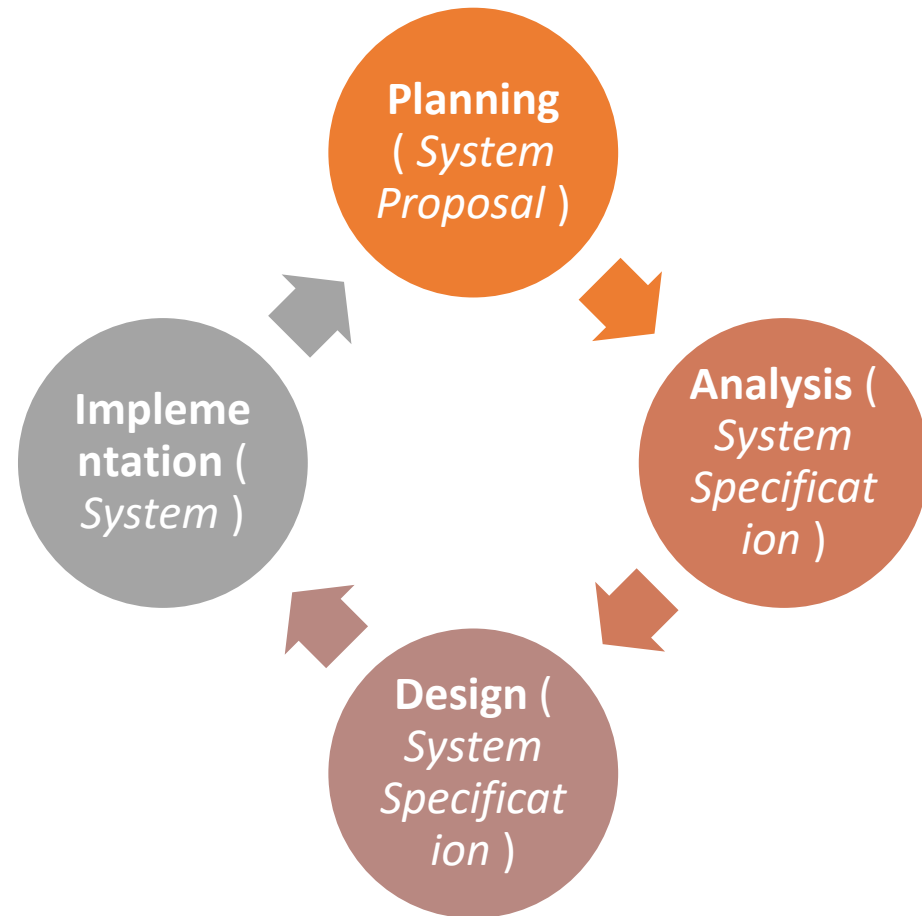
Planning
(*System Proposal*)

Analysis and Design
(*System Specification*)

Implementation
(*Software*)

Maintenance
(*Updated Software*)

Software development life cycle



1. **Planning** : Why Software Should Be Developed?

- System request
- Feasibility Analysis (project size estimation)

2. **Analysis** : Who are the Users and Software Workflow

- Requirements gathering
- Business process modeling

3. **Design** : What is the composition of the software?

- Program design
- User interface design
- Data design

4. **Implementation** : Software

- System construction
- Testing
- Documentation
- Installation

(Tilley, 2012) (Dennis, 2016) (Valacich, 2017)

Software Development Methodology

- A formalized **approach to implementing** the Software Development Life Cycle (*Dennis, 2012*)
- A **simplified representation** of a software process (*Sommerville, 2015*)
- A distinct **set of activities** , actions, tasks, milestones, and work products required to engineer high quality software (*Pressman, 2015*)

Software Development Methodology

1. Structured Design (Prescriptive) (1967-)

- Waterfall method
- Parallel development

More
Prescriptive/
Documentation

2. Rapid Application Development (Iterative) (1985-)

- Phased Development
- Prototyping

3. Agile Development (Adaptive) (1995-)

- Extreme Programming (XP)
- Scrum

More
Adaptive/
Communication

System Request: Sistem Penjualan Musik Online

Project Sponsor: Margaret Mooney, Vice President of Marketing

Business Needs: Project ini dibangun untuk:

1. Mendapatkan pelanggan baru lewat Internet
2. Meningkatkan penjualan dari pelanggan baru
3. Meningkatkan penjualan dari pelanggan lama
4. Mengurangi biaya operasional dan telepon

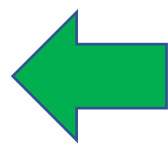
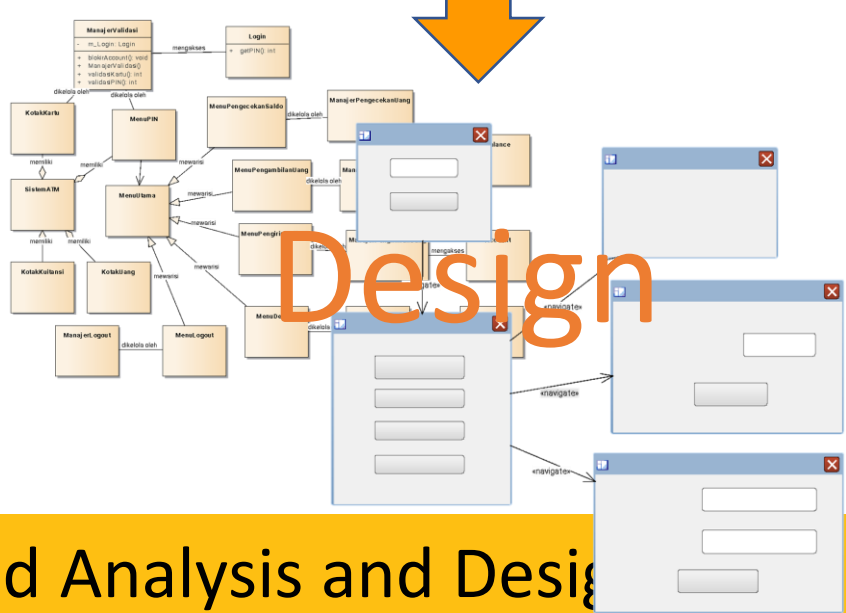
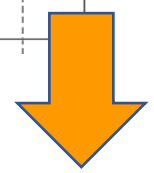
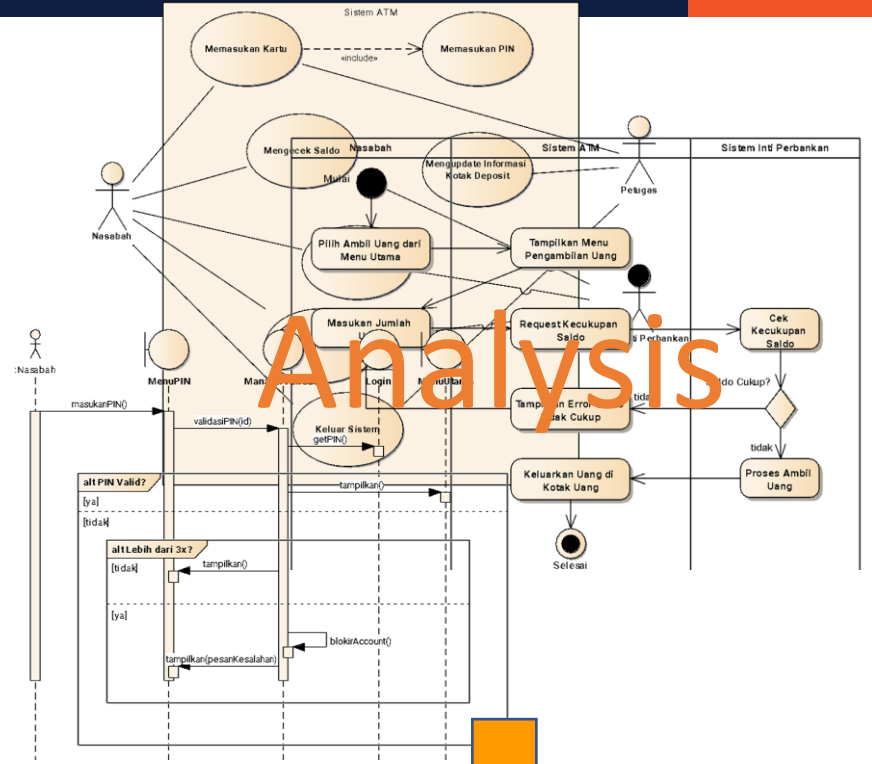
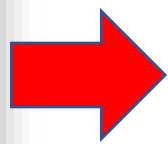
Studi Kelayakan Sistem Penjualan Musik Online

Business Requirement: Margaret Mooney dan Alec Adams membuat studi kelayakan untuk pengembangan Sistem yang mendukung Sistem Penjualan Musik Online

Kelayakan Teknis

Sistem penjualan musik online layak secara teknis, meskipun memiliki beberapa risiko.

	2016	2017	2018
Peningkatan penjualan dari pelanggan baru	0	400,000,000	500,000,000
Peningkatan penjualan dari pelanggan lama	0	600,000,000	700,000,000
Pengurangan biaya operasional dan telepon	0	100,000,000	100,000,000
Total Benefits:	0	1,100,000,000	1,300,000,000
PV of Benefits:	78,950,084	1,091,005,198	1,076,501,132
PV of All Benefits:	78,950,084	1,091,005,198	1,076,501,132
Honor Tim (Planning, Analysis, Design and Implementation)	360,000,000	0	0
Honor Konsultan Infrastruktur Internet	90,000,000	0	0
Total Development Costs:	450,000,000	0	0
Honor Pengelola Web	60,000,000	70,000,000	80,000,000
Biaya Lisensi Software	50,000,000	60,000,000	70,000,000
Hardware upgrades	100,000,000	100,000,000	100,000,000
Biaya Komunikasi	20,000,000	30,000,000	40,000,000
Biaya Marketing	100,000,000	200,000,000	300,000,000
Total Operational Costs:	330,000,000	460,000,000	590,000,000
Total Costs:	780,000,000	460,000,000	590,000,000
PV of Costs:	735,849,057	409,398,362	495,375,377
PV of all Costs:	735,849,057	1,145,247,419	1,640,622,796
Total Project Costs Less Benefits:	-780,000,000	640,000,000	710,000,000
Yearly NPV:	-735,849,057	596,597,722	596,129,691
Cumulative NPV:	-735,849,057	-166,251,335	429,878,356
Return on Investment (ROI) di Tahun 3:	26.2%	0.262021445	0.262021445
Break-even Point (BEP):	2.28 tahun	2.278884507	2.278884507



Implementation



Source : Unified Modeling Language,
Romi Satrio Wahono
<http://romisatriawahono.net>

System Request: Sistem Penjualan Musik Online

Project Sponsor: Margaret Mooney, Vice President of Marketing

Business Needs: Project ini dibangun untuk:

1. Mendapatkan pelanggan baru lewat Internet
2. Meningkatkan penjualan dari pelanggan lama
3. Fitur Pencarian Produk
4. Fitur Pemesanan Produk

Studi Kelayakan Sistem Penjualan Musik Online

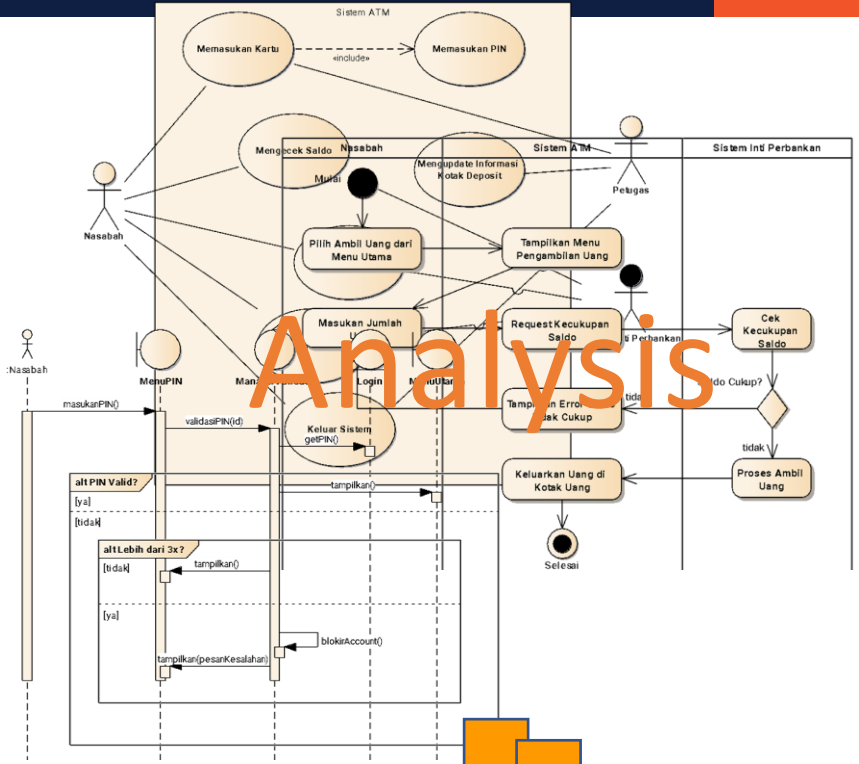
Business Requirement: Margaret Mooney dan Alec Adams membuat studi kelayakan untuk pengembangan Sistem yang mendukung Sistem Penjualan Musik Online

Kelayakan Teknis

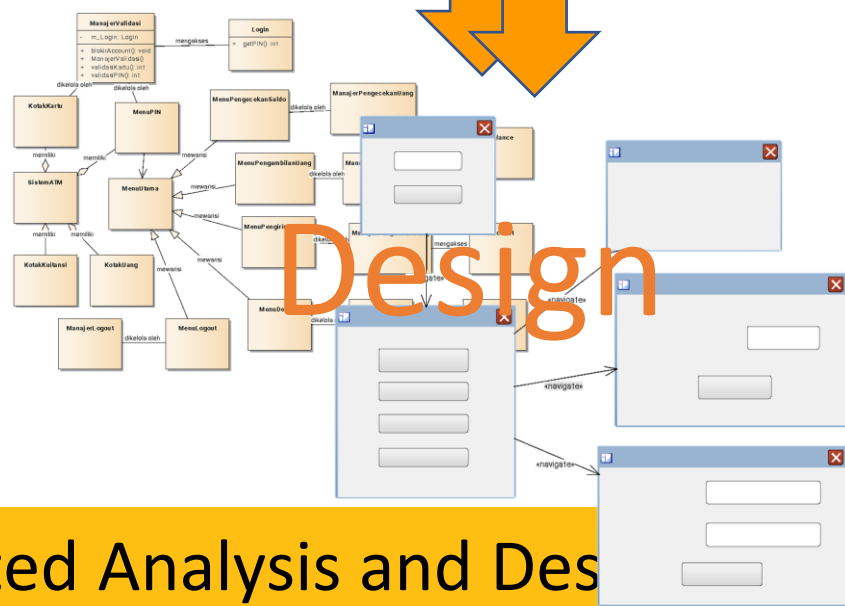
Sistem penjualan musik online layak secara teknis, meskipun memiliki beberapa risiko.

	2016	2017	2018
Peningkatan penjualan dari pelanggan baru	0	400,000,000	500,000,000
Peningkatan penjualan dari pelanggan lama	0	600,000,000	700,000,000
Pengurangan biaya operasional	0	100,000,000	100,000,000
Total Benefits:	0	1,100,000,000	1,300,000,000
PV of Benefits:	78,951,088	1,091,095,368	
PV of All Benefits:	78,951,088	1,091,095,368	1,071,501,312
Honor Tim (Planning, Analysis, Design and Implementation)	360,000,000	0	0
Honor Konsultan Infrastruktur Internet	90,000,000	0	0
Total Development Costs:	450,000,000	0	0
Honor Pengelola Web	60,000,000	70,000,000	80,000,000
Biaya Lisensi Software	50,000,000	60,000,000	70,000,000
Hardware upgrades	100,000,000	100,000,000	100,000,000
Biaya Komunikasi	20,000,000	30,000,000	40,000,000
Biaya Marketing	100,000,000	200,000,000	300,000,000
Total Operational Costs:	330,000,000	460,000,000	590,000,000
Total Costs:	780,000,000	460,000,000	590,000,000
PV of Costs:	735,849,057	409,398,362	495,375,377
PV of all Costs:	735,849,057	1,145,247,419	1,640,622,796
Total Project Costs Less Benefits:	-780,000,000	640,000,000	710,000,000
Yearly NPV:	-735,849,057	569,507,722	596,129,691
Cumulative NPV:	-735,849,057	-166,251,335	429,878,356
Return on Investment (ROI) di Tahun 3:	26.2%		0.262021445
Break-even Point (BEP):	2.28 tahun		2.278884507

Planning



Analysis



Design

Version 1

Version 2

MusicPedia

Update Song

Title:

Album:

Band or Artist's Name:

Genre:

Track Number:

Year:

Album Cover:

Upload Song:

Harga:

Implementation

Source : Unified Modeling Language,
Romi Satrio Wahono
<http://romisatriawahono.net>

System Request: Sistem Penjualan Musik Online

Project Sponsor: Margaret Mooney, Vice President of Marketing

Business Needs: Project ini dibangun untuk:

1. Mendapatkan pelanggan baru lewat Internet

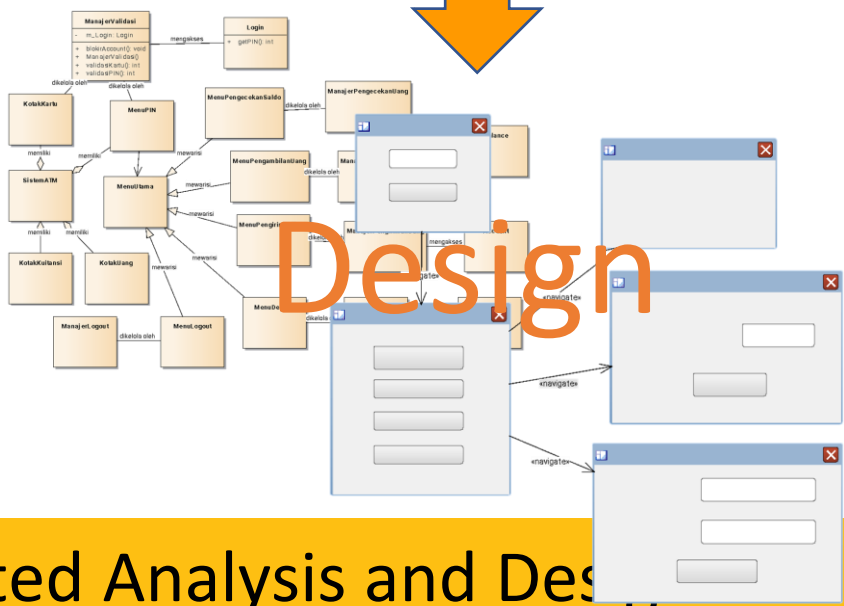
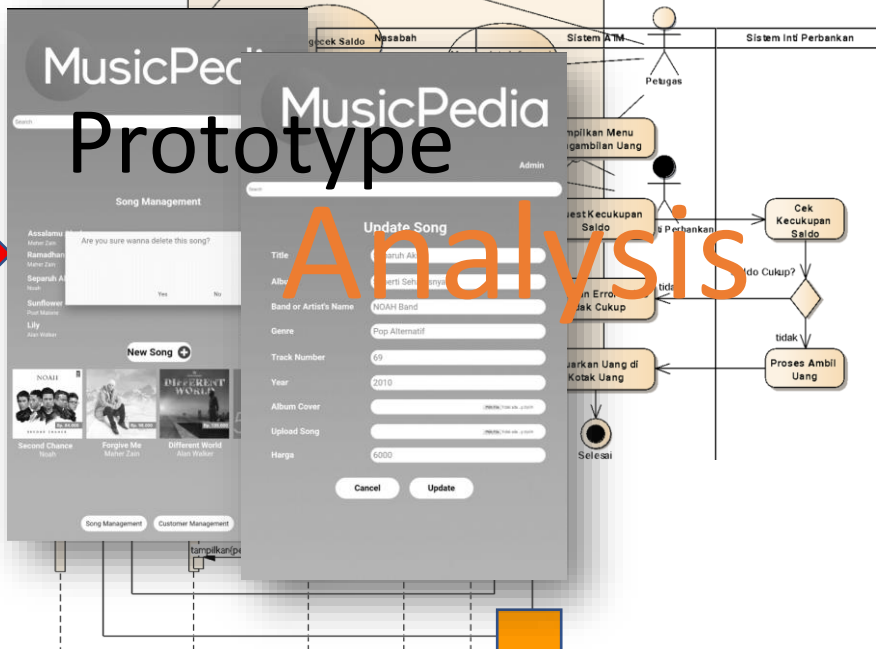
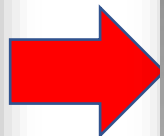
Studi Kelayakan Sistem Penjualan Musik Online

Margaret Mooney dan Alec Adams membuat studi kelayakan untuk pengembangan Sistem yang mendukung Sistem Penjualan Musik Online

Kelayakan Teknis

Sistem penjualan musik online layak secara teknis, meskipun memiliki beberapa risiko.

	2016	2017	2018
Peningkatan penjualan dari pelanggan baru	0	400,000,000	500,000,000
Peningkatan penjualan dari pelanggan lama	0	600,000,000	700,000,000
Pengurangan biaya operasional di telepon	0	100,000,000	100,000,000
Total Benefits:	0	1,100,000,000	1,300,000,000
PV of Benefits:	78,951,081	91,091,091	91,091,091
PV of All Benefits:	170,032,262	1,009,909,091	1,009,909,091
Honor Tim (Planning, Analysis, Design and Implementation)	360,000,000	0	0
Honor Konsultan Infrastruktur Internet	60,000,000	0	0
Total Development Costs:	420,000,000	0	0
Honor Pengelola Web	80,000,000	70,000,000	80,000,000
Biaya Lisensi Software	50,000,000	60,000,000	70,000,000
Hardware upgrades	100,000,000	100,000,000	100,000,000
Biaya Komunikasi	20,000,000	30,000,000	40,000,000
Biaya Marketing	100,000,000	200,000,000	300,000,000
Total Operational Costs:	330,000,000	460,000,000	590,000,000
Total Costs:	780,000,000	460,000,000	590,000,000
PV of Costs:	735,849,057	409,398,362	495,375,377
PV of all Costs:	735,849,057	1,145,247,419	1,640,622,796
Total Project Costs Less Benefits:	-780,000,000	640,000,000	710,000,000
Yearly NPV:	-735,849,057	569,507,722	596,129,691
Cumulative NPV:	-735,849,057	-166,251,335	429,878,356
Return on Investment (ROI) di Tahun 3:	26.2%	0.262021445	0.262021445
Break-even Point (BEP): 2.28 tahun	2 + (596,129,691 - 429,878,356) / 596,129,691	2.278884507	2.278884507



Implementation

MusicPedia Admin interface showing the 'Update Song' form and a list of songs.



Source : Unified Modeling Language,
Romi Satrio Wahono
<http://romisatriawahono.net>

Object Oriented Software Development Paradigm

Paradigms in Software Development

- **Conventional Paradigm**

- Focus on **Process** (Input-Process-Output);
- Data is **separate** from Procedure;
- **Decomposition Functional** .

- **Object Oriented Paradigm**

- Focus on **Domain Objects** , **not** procedures ;
- **Data** and **Procedures** are stored in **Objects** ;
- **Decomposition Data/Object**

Conventional Paradigm

- The conventional paradigm refers to **the strategy functional (algorithm)** based **decomposition** or event/process/sub-process partitioning.
- This paradigm covers the entire system development process in its entirety, including:
 - Structured analysis (*structured analysis* , DeMacro, 1978).
 - *Structured* design (Yourdon & Constantine, 1979).
 - SA/SD (*Structured Analysis and Structured Design* , Yourdon 1989).
 - **Functional Decomposition = Functions + SubFunctions + Function Interfaces.**
- Function decomposition requires a great deal of system analyst intelligence to map the problem domain into functions and subfunctions. The problem is that functions are highly volatile.

Object Oriented Paradigm

- The object-oriented paradigm is a perspective that is not just an algorithm **based** on **objects** where everything is seen as **an object** .
- This paradigm covers the entire life cycle in systems development.
 - Object Oriented Design (Booch 1991).
 - Object Oriented Analysis and Design (Coad & Yourdon 1989).
 - Object Modeling Technique (OMT) (Rumbaugh 1991).
 - Object-oriented Systems Analysis (Shlaer & Mellor 1988)
 - Object-oriented Systems Development (Henderson-Sellers 1992).
- Coad-Yourdon Definition:
Object oriented = objects + classification + inheritance + communication
- Booch's Definition:
Object-oriented = abstraction (objects) + encapsulation + modularity + hierarchy + [typing+concurrency+persistence].

Object Oriented Paradigm

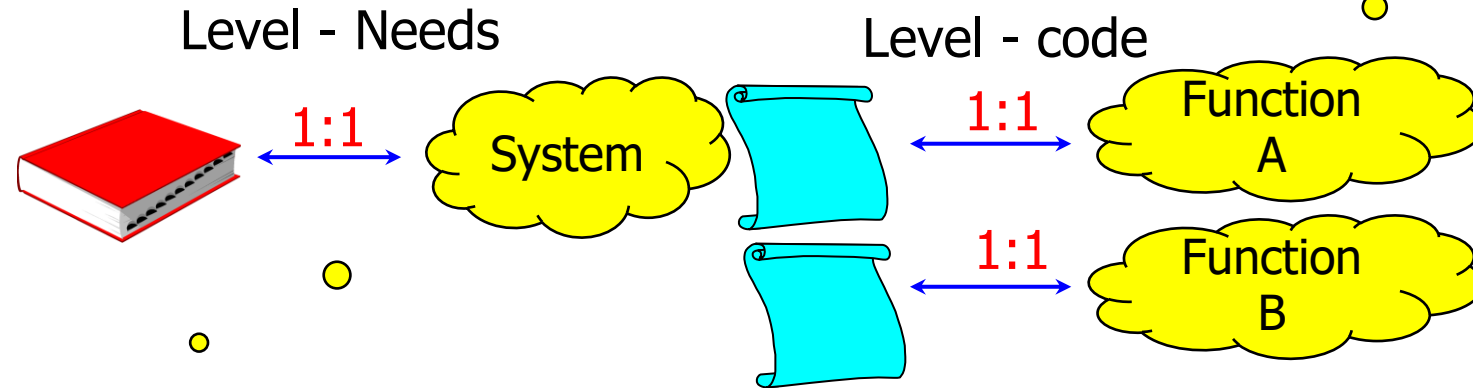
- Object-Oriented Methods provide a set of techniques for analyzing, decomposing and modularizing the architecture of software systems.
- Object-Oriented Methods are characterized by **structuring architecture object** and **class based systems** .

Importance of its use:

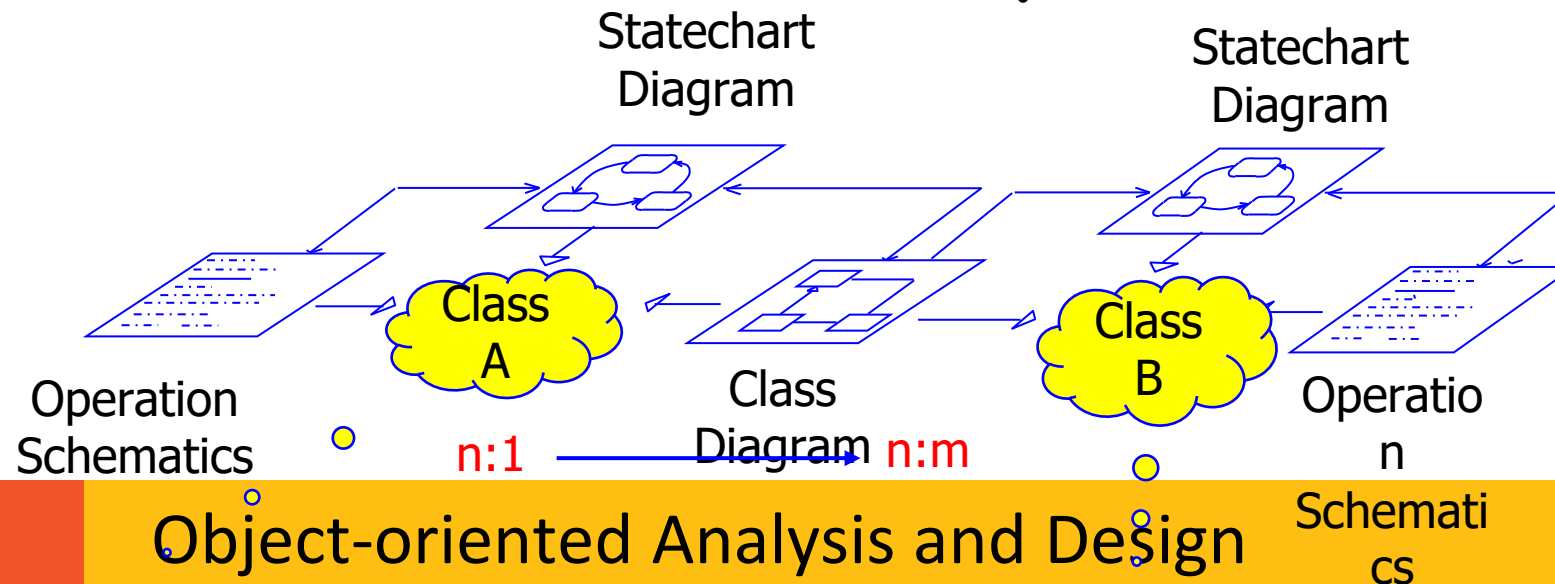
- Generally systems evolve and their functions change, but objects and classes remain stable over time.
- Used for large sized systems.
- Used for systems that change frequently.

Object Oriented vs Conventional Paradigm

1. Conventional Paradigm



2. Object Oriented Paradigm



Object Oriented vs Conventional Paradigm



The conventional paradigm focuses on:

- Decomposition of the system into a number of functions in a hierarchical structure (**modules**).
- Process and Procedure of each module.

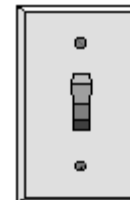
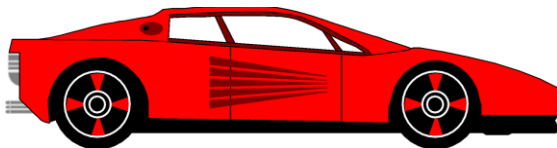
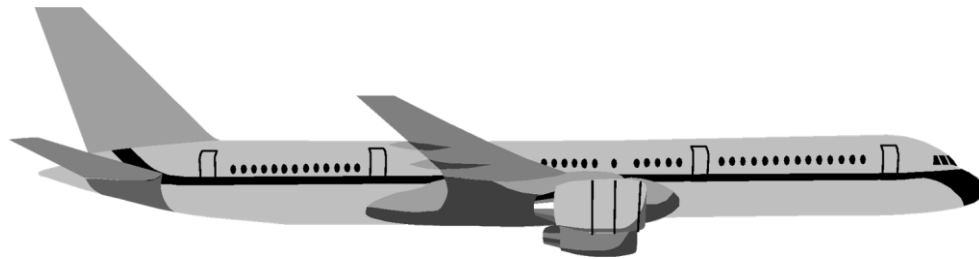
The object-oriented paradigm focuses on:

- Real-world objects that work together (not hierarchical).
- Models of real-world objects (**scenarios**).
- The role of these objects in describing the software system (**method**).

Objects

Usually in the form of an object or an event

- **Concrete objects:** airplanes, lamps, books, ...
- **Concept:** Shopping cart, lecture, ...
- **Abstract:** company, business, school, ...

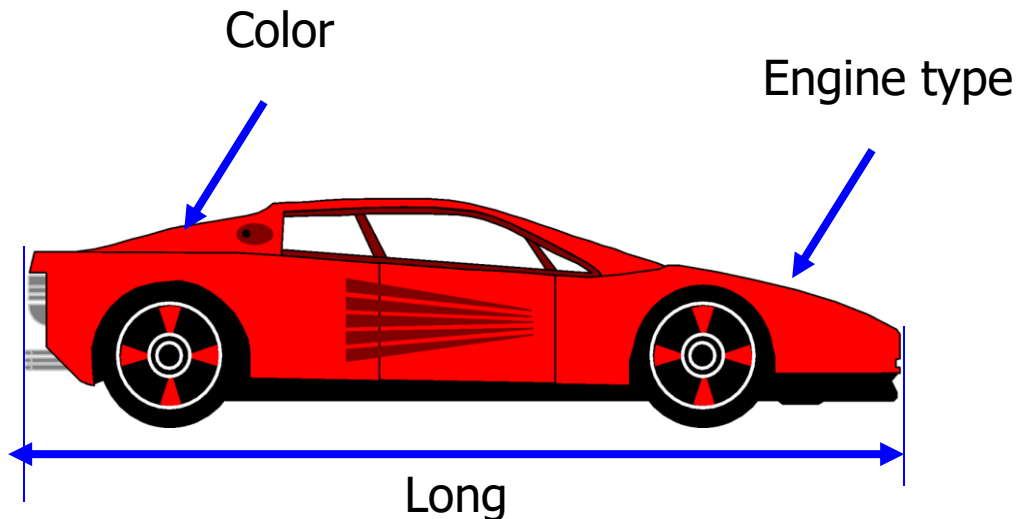


Object > Attributes

ATTRIBUTE:

It is an internal value or related data on an object that shows:

- Characteristics or properties of an object
- Depiction of the state *of* an object



Object name: Car

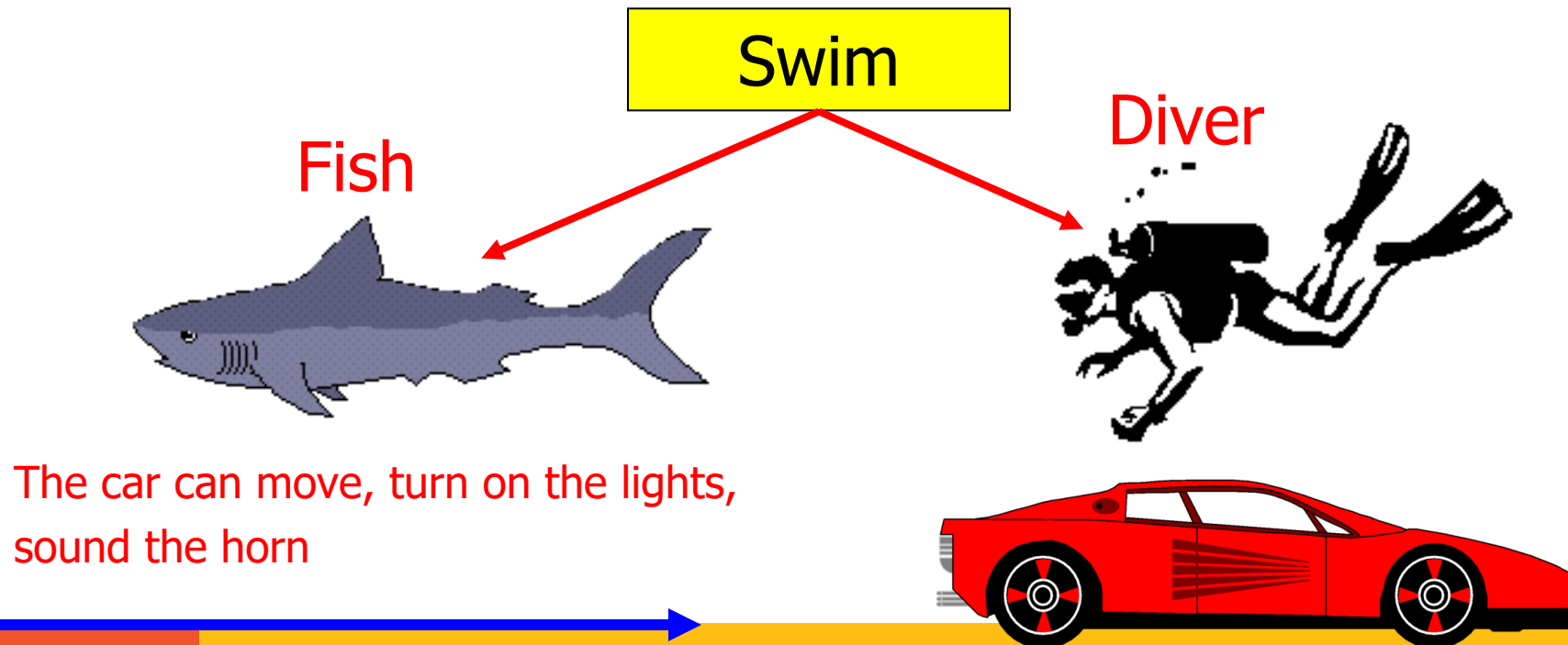
Attribute:

- Brand : Toyota
- Cylinder : 2000 cc
- Red
- Status :
New/Running

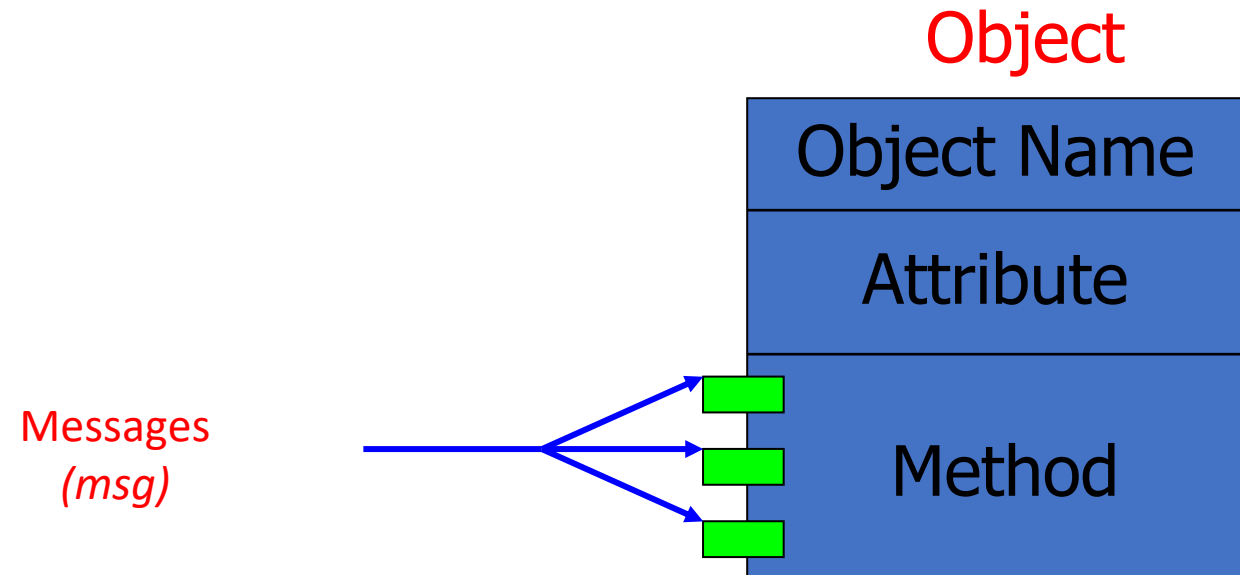
Object > Method/Behaviour/Function



- Behavior defines how an object acts and reacts, and relates to the functions applied to an attribute.
- The behavior of an object is called a method or service **operation**.



Objects



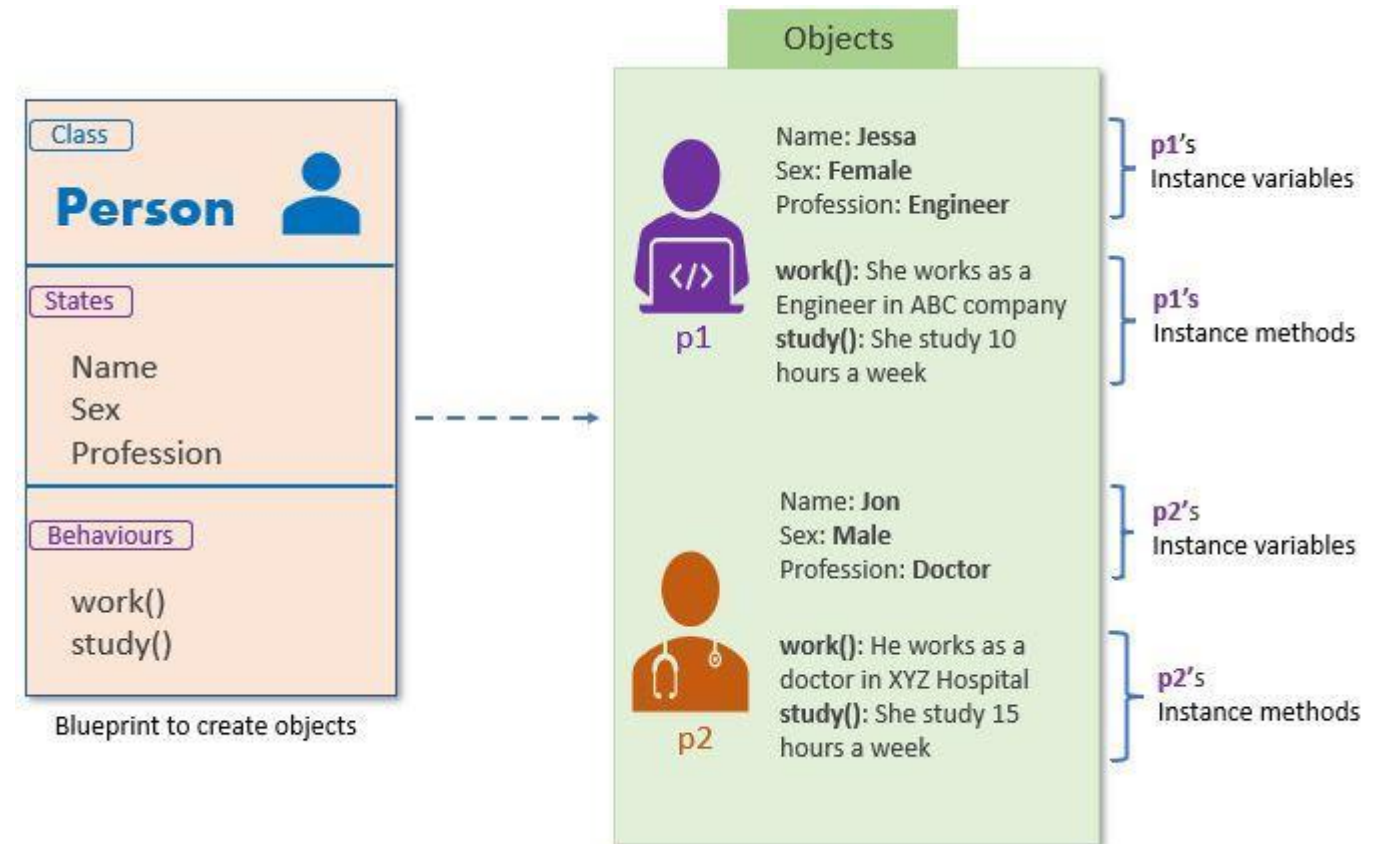
- **Objects** is a collection of attributes (data) together with a combination of methods (functions) used to operate those attributes.

Object = Attributes + Methods

- The outside world communicates to objects by sending messages. (*message*).

Class vs Object

- Class is **the design** of an object, and an object is the result of **instantiating** a class.



Inheritance

- Inheritance of traits, from more general classes to more specific classes
- Also called **generalization** or **is- a relation**

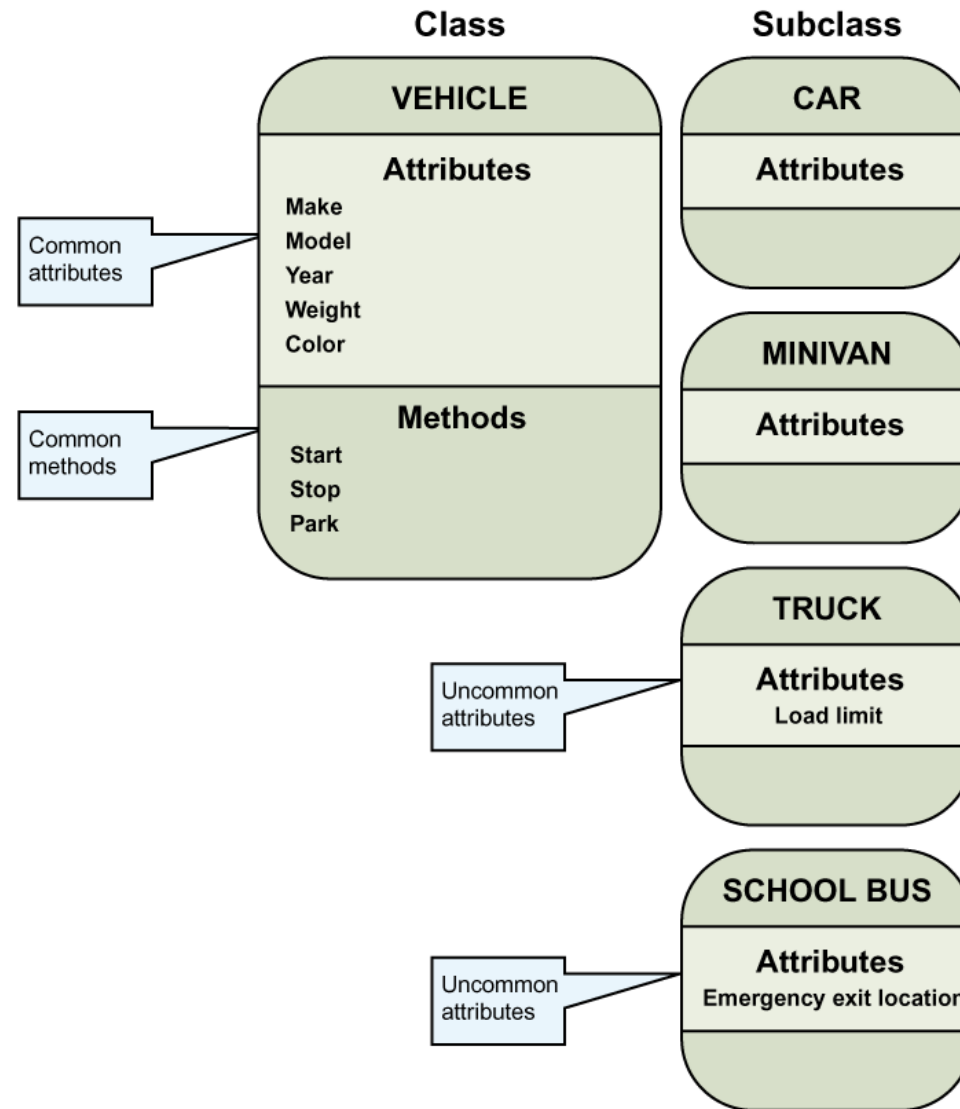


FIGURE 5-17

The VEHICLE class includes common attributes and methods. CAR, TRUCK, MINIVAN, and SCHOOL BUS are instances of the VEHICLE class.

Domain Modeling

Understanding Domain Modeling

- In the previous semester, you created an **application** , which started from a **design. class**
- **Design** The **class** is usually depicted in a **diagram. class** (in this course, class diagrams will be discussed in more detail)
- **analysis** is a core activity that greatly influences the subsequent program coding model.
- So, where do we start the analysis and identification of these classes?

Understanding Domain Modeling

- **Domain Modeling** is a technique for identifying objects through words. items on the list requirements that are classified in the same problem area (domain) to be used as candidates class or candidate object
- **Domain The model** is the foundation of the static part of a system model.
- **Point The beginning** of starting domain modeling is from the requirements or system needs of the user/client.
- Collecting a good list of requirements will result in a good domain model.

Collecting a Needs List



- You can use a model diagram that can describe **needs** and **relationships** with other needs, for example a **Mind diagram. Map** . Mind maps can help improve more detailed needs tracking.
- Focus on functional **requirements** first. The use of dysfunctional requirement statements that do not describe the function of a system can be limited first
- **Avoid** using **language technical** such as return, parameter, void, int etc. so that the requirement statement can be easily read.
- **Scenary test cases** for each need can be identified at once, so that the resulting needs can be tested.
- **Classify needs** into different types to form problem domains.
- Treat every **need** you get as something very important
- Try your best and **focus** on the **exploration process needs** . Especially if you meet a client who is not tech-savvy. We need to try to guide and trigger so that the client's needs can be conveyed.

Rule Domain Modeling

- Domain model analysis must **focus** on real world **objects** contained in a domain or problem area
- In the early stages, the results of the domain model may **not be final** , and will certainly **evolve** along with software engineering activities (domain model updates). So **it is not need too long time** in this early stage
- For the initial model domain, make sure the analysis is focused on the **problem domain** (area) . Do not mix it with classes related to the user interface.
- If it is necessary to describe the relationship between objects, try to use class relationship patterns (inheritance, composition, aggregation, etc.)

Help!



I want to create an online bookstore, but I don't know what an online bookstore should do.

Can you help me to find out the abilities that an online bookstore should have?

High Level Requirements from Previous Meeting (1)

- The bookstore must be a web-based application
- The user must be able to create a customer account
- The system shall maintain a list of accounts in database
- When a user logs in, his/her username and password must always be matched against the password in the master account list
- The bookstore must be able to sell books with orders accepted over the internet
- The user must be able to add books into an online shopping chart, prior to checkout (vis versa)
- The books should be stored in master catalogue in database system

High Level Requirements from Previous Meeting (2)

- The user is able to add books into a wish list
- The user must be able to pay by credit card, digital wallet, or other payment channel (through convenience store)
- The user must be able to search books by various search methods – by title, author, keyword, or category – and the view the book's details
- The user must be able to post a book's review. It should contain a customer rating (1-5)
- Book reviews must be moderated by staff before published on the website
- The customer may click to view full review on a separate page

Can you find the “objects”? (1)

- The **bookstore** will be web based
- The **user** must be able to create a **customer account**
- The **system** shall maintain a **list of accounts** in database
- When a user logs in, his/her **username** and **password** must always be matched against the password in the **master account list**
- The bookstore must be able to sell **books** with **orders** accepted over the **internet**
- The user must be able to add books into an online **shopping chart** , prior to **checkout** (vis versa)
- The books should be stored in **master catalog** in database system

Can you find the “objects”? (2)

- The user is able to add books into a **wish list**
- The user must be able to pay by **credit card** , **digital wallet** , or **other payment channel** (through convenience store)
- The user must be able to search books by various **search methods** – by **title** , **author** , **keyword** , or **category** – and the view the **book's details**
- The user must be able to post a **book's review** . It should contain a **customer rating** (1-5)
- **Book reviews** must be moderated by staff before published on the website
- **The customer** may click to view full review on a separate page

List the “objects” first

- Books
- Bookstore
- Checkout
- Customer Account
- List of accounts
- Master account list
- Order
- Password
- Shopping chart
- System
- User
- Username
- Author
- Book Detail
- Book Review
- Category
- Credit card
- Customer
- Customer Rating
- Digital wallet
- Keyword
- Other payment channels (convenience stores)
- Search method
- Title
- Wish list

Let's remove the “unnecessary object”

- Book
- ~~Bookstore~~
- Checkout
- Customer Account
- List of accounts
- Master account list
- Master catalog
- Order
- ~~Password~~
- Shopping chart
- ~~System~~
- User
- Username
- Author
- Book Detail
- Book Review
- Category
- Credit card
- Customer
- Customer Rating
- Digital wallet
- ~~Keyword~~
- Other payment channels (convenience stores)
- Search method
- ~~Title~~
- Wish list

Updated List of “Object”

- Book
- Checkout
- Customer
- Customer account
- Master account list
- Master catalog
- Order
- Shopping chart
- Author
- Book review
- Category
- Credit card
- Customer rating
- Digital wallet
- Other payment channels
- Search method
- Wish list

So, what is the “ **object** ” then?

Why do we get rid of some of “objects” in our list?

The Objects : *Psss! We make it simple for now* (1)

- Objects are "something" that **interact** to achieve software the system objectives
- The objects **usually mimic real-world objects**
- The real-word objects may **have physical present** or just represent **well-understood conceptual entities** in the application.

Examples:

- Book (physical)
- Book review (conceptual)

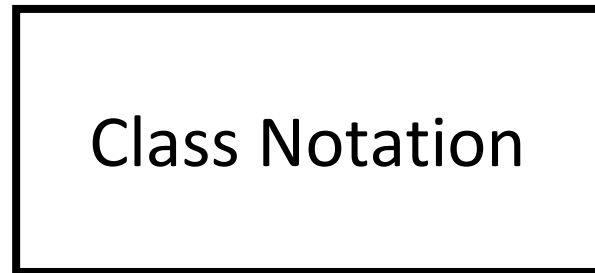
The Objects : *Psss! We make it simple for now* (2)

- Objects are characterized by having both **attribute** (**state**) and **behavior**
- The attribute is the information of the object. For example:
 - Title, ISBN, publisher, so on
- The behavior is something that the object can do (action). For example:
 - Give an information about book title (get title)
 - Set the ISBN
 - etc
- The object is an instance of the class (we will talk about class later)

Revisit

- Book
- ~~Bookstore~~
- Checkout
- List of accounts
- Master account list
- Master catalog
- Order
- ~~Password~~
- Shopping chart
- System
- User
- ~~Username~~
- Author
- Book Detail
- Book Review
- Category
- Credit card
- Customer
- Customer Rating
- Digital wallet
- ~~Keyword~~
- Other payment channels (convenience stores)
- Search method
- ~~Title~~
- Wish list

Determine the relationships of the objects



This is the object



Generalization



“Is-a” relationship



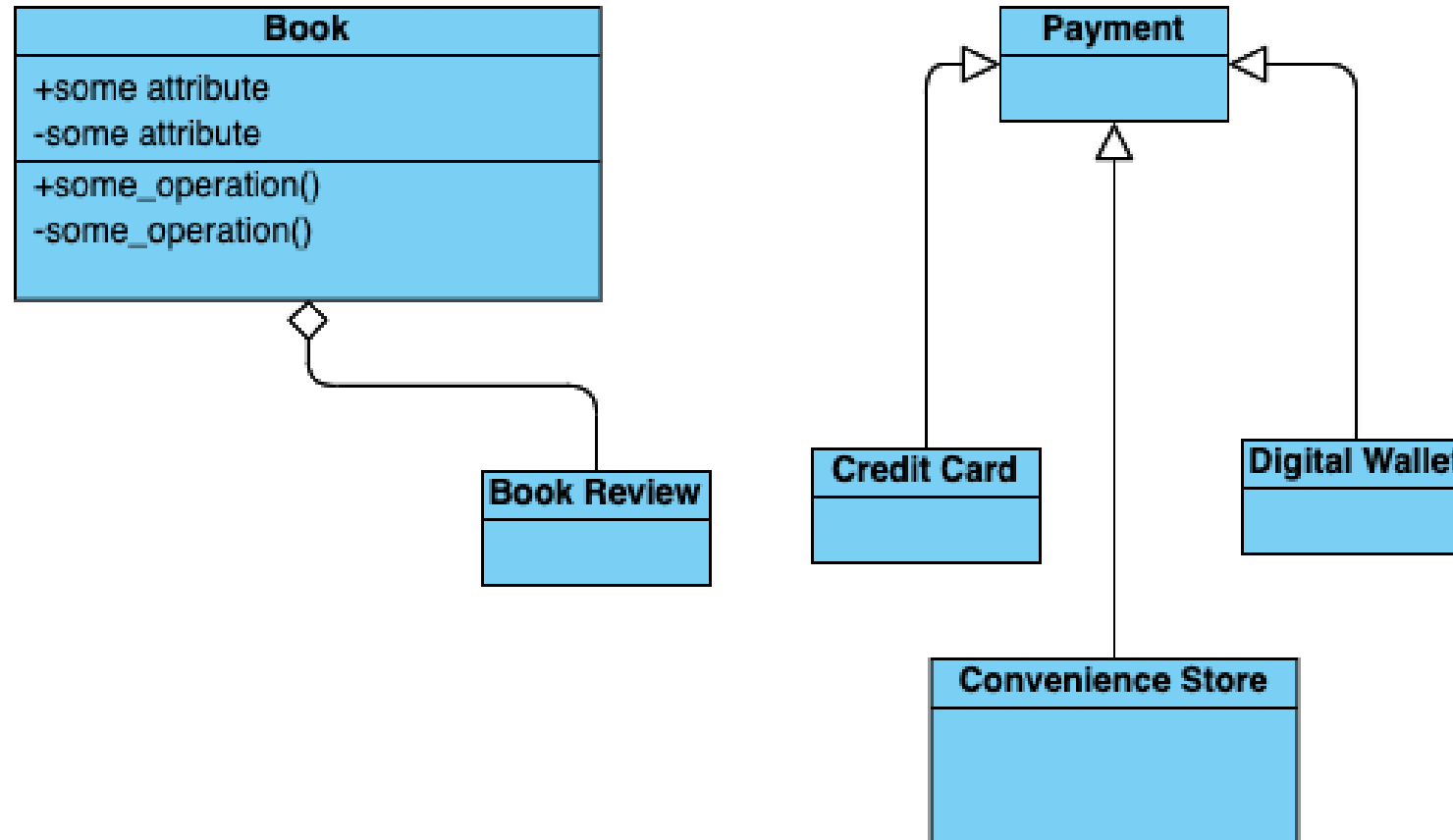
Aggregation



“has-a” relationship

Examples

Visual Paradigm Online Free Edition



Visual Paradigm Online Free Edition



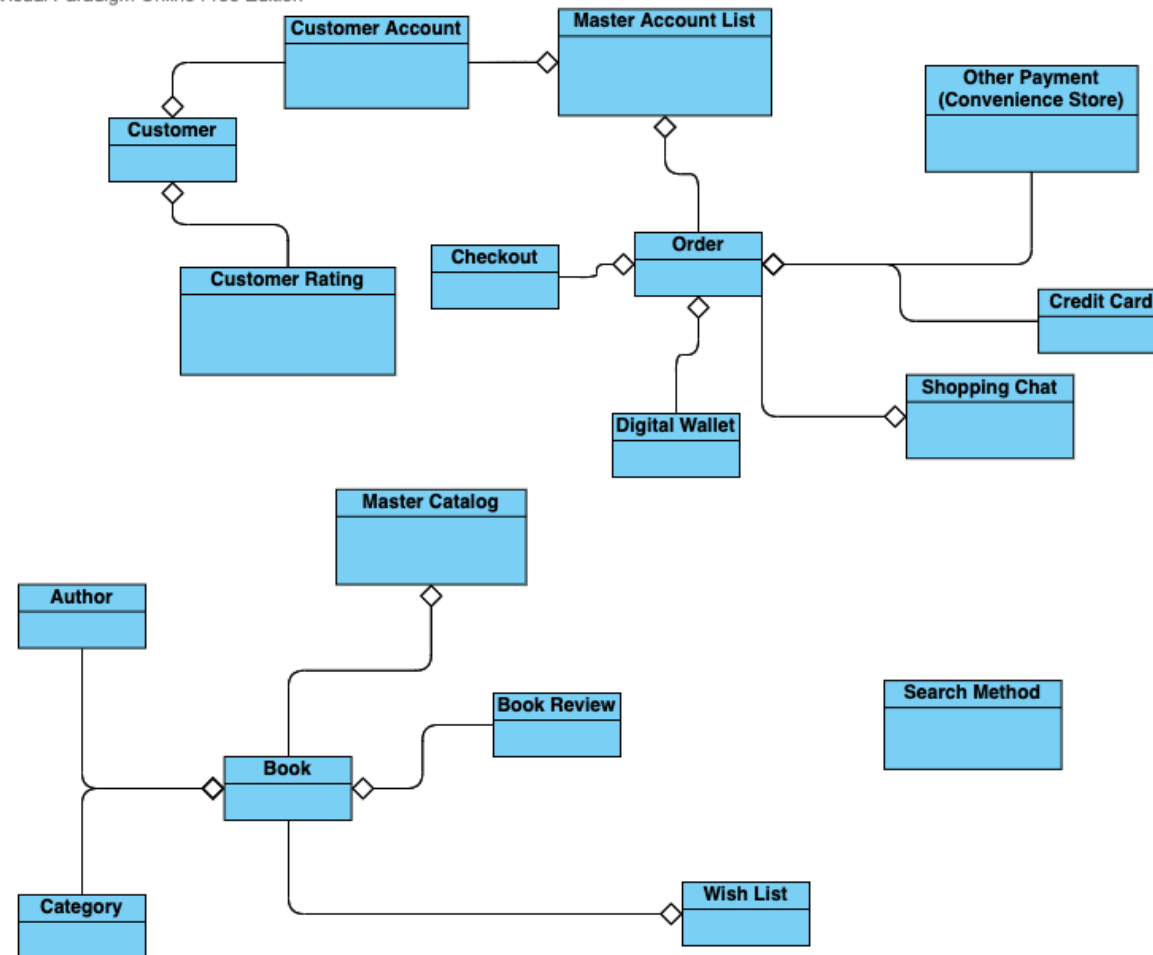
Discussion Time !

According to your requirements, please define the objects and determine the relationship between them.

Duration : **30 minutes**

1st attempt domain model

Visual Paradigm Online Free Edition



Visual Paradigm Online Free Edition

Homework Task

- Please refine your domain model in order to make your object identification clearer (at least for now 😊)
 - You can identify the new object(s)
 - You can refine the relationship

Next?

Use Case Diagram

